

Technical Report

Autonomous Visual Intelligence System for Scene Understanding

BSCS — Department of Computer Science

Student Name: Muhammad Ali Waseem Roll No: F2022376029

1. Executive Summary

In this project I implemented a multi-stage computer vision pipeline for outdoor scene understanding in an autonomous-robotics context. I built the pipeline in four stages. In stage one I detect road boundaries and structural lines using Gaussian smoothing, Canny edge detection, and the Probabilistic Hough Line Transform. In stage two I establish feature correspondence between two views of the same landmark using SIFT keypoints and descriptors filtered with Lowe's ratio test. In stage three I construct a panorama from three overlapping frames using SIFT matching, RANSAC homography estimation, progressive warping, and alpha blending — which I implemented manually without cv2.Stitcher. In stage four I classify outdoor scenes into four land-cover categories using transfer learning with MobileNetV2, and I interpret selected predictions using Grad-CAM.

I executed the system end-to-end in Google Colab. I reached a validation accuracy of 92% on the classification task (the target was 70%), a RANSAC inlier ratio above 0.94 on both stitching pairs, and 211 good SIFT matches on the landmark pair. My pipeline combines fast, interpretable classical methods (Tasks 1–3) with a learning-based classifier (Task 4), so the robot can reason about both geometric structure and semantic scene category.

2. Problem and Proposed Solution

Task	Problem question	Proposed solution
T1	How can the robot detect road boundaries, lane markings, or structural lines?	I convert to grayscale, smooth with a 5×5 Gaussian kernel ($\sigma=1.2$), compare three Canny threshold pairs, then apply the Probabilistic Hough Transform and overlay the detected lines.
T2	How can the robot recognize the same landmark from a different viewpoint?	I detect SIFT keypoints/descriptors, match with FLANN, filter with Lowe's ratio test at 0.75, and visualize the top 50 reliable matches.
T3	How can multiple overlapping frames be combined into a wider panoramic map?	I match SIFT features, estimate homographies with RANSAC, progressively warp frames into a shared canvas, and use alpha blending to reduce seams.
T4	How can the robot classify the overall outdoor scene category?	I fine-tune a MobileNetV2 transfer-learning classifier with data augmentation, then evaluate with validation accuracy, confusion matrix, precision/recall, and Grad-CAM.

3. Pipeline Architecture Diagram

My pipeline processes input outdoor images through four processing stages, each producing outputs that feed my final critical evaluation:

Input outdoor images

```
|
+--> Task 1:  Gaussian smoothing -> Canny edges -> Hough lines -> line overlay
|
+--> Task 2:  SIFT keypoints -> descriptor matching -> Lowe ratio filter -> matches
|
+--> Task 3:  SIFT matches -> RANSAC homography -> warping -> blending -> panorama
|
+--> Task 4:  resized images -> augmentation -> MobileNetV2 -> class + Grad-CAM
|
+--> Task 5:  quantitative metrics + failure analysis + trade-off discussion
```

4. Datasets and Image Sources (Citations)

All images I used in this project are sourced externally and cited below. I did not self-capture any images.

Task	Data used	Source / Citation
T1	One outdoor road image (Murree road, Pakistan).	Photograph sourced from Unsplash (free-to-use license). unsplash.com
T2	Two views of the same landmark — Wazir Khan Mosque, Lahore.	Photographs by Danish Jaffry, Unsplash (free-to-use license). unsplash.com
T3	Three overlapping frames of a nature/landscape scene.	Sample overlapping sequence from the open-source repository Avinash793/panoramic-image-stitching, GitHub. (Only the images were used; the stitching code is the author's own implementation.)
T4	600 images (150 per class) across 4 land-cover categories: road (Highway), building (Residential), vegetation (Forest), water (River).	EuroSAT dataset (Sentinel-2 satellite land-cover imagery), Helber et al., 2019. tensorflow.org/datasets/catalog/eurosat

Note on Task 4 data: EuroSAT consists of aerial/satellite land-cover patches rather than ground-level photographs. The four categories are therefore land-cover classes (Highway, Residential, Forest, River), which I relabelled as road, building, vegetation, and water to match the robotics scene-understanding framing. The assignment permits any four outdoor categories.

5. Quantitative Results

The metrics below were produced directly by my Colab run and saved as CSV/txt outputs.

Task 1 — Edge Detection & Hough Transform

Gaussian kernel / sigma	5×5, $\sigma = 1.2$
Selected Canny thresholds	low = 50, high = 150
Hough lines detected (final)	43 lines

Hough parameter experiments ($\rho=1$, $\theta=\pi/180$):

threshold	minLineLength / maxLineGap	detected lines
40	40 / 10	49
60	60 / 20	43
90	90 / 30	27

My observation: a lower Hough threshold with short minimum line length accepts more (and noisier) segments; raising the threshold returns fewer but more stable lines. I found the 50/150 Canny pair with threshold 60 gave the cleanest lane/structure overlay.



Task 2 — SIFT Feature Extraction & Matching

Keypoints — Image 1	480
Keypoints — Image 2	410
Raw descriptor matches	480
Good matches (Lowe 0.75)	211
Match quality ratio	0.44

Effect of Lowe's ratio threshold (precision vs. recall):

ratio threshold	good matches	match quality ratio
0.60	152	0.317

ratio threshold	good matches	match quality ratio
0.70	185	0.385
0.75	211	0.440
0.85	262	0.546
0.95	356	0.742

A smaller ratio (e.g. 0.60) keeps fewer but more reliable matches (higher precision, lower recall); a larger ratio (0.95) admits many more matches but includes ambiguous/false correspondences (higher recall, lower precision). The 0.75 default balances the two.



Task 3 — Panorama Construction (SIFT + RANSAC)

Stitching pair	RANSAC inlier ratio
panorama + image 2	0.951
panorama + image 3	0.943

Intermediate homography matrices (mapping each new image into the panorama frame):

After image 2:

```
[[ 1.00184314e+00  2.83796245e-05  1.93644918e+02 ]
 [ 3.23778768e-04  1.00050648e+00 -4.60323407e-02 ]
 [ 4.71336087e-07  5.91969470e-07  1.00000000e+00 ]]
```

After image 3:

```
[[ 1.00033510e+00 -1.16322378e-04  3.22828842e+02 ]
 [ 6.03571307e-04  1.00046776e+00 -5.68082047e-02 ]
 [ 1.42911991e-06  2.69357779e-07  1.00000000e+00 ]]
```

Both inlier ratios I obtained exceed 0.94, which indicates strong geometric agreement between frames and a well-conditioned homography. The near-identity rotation/scale blocks with large horizontal translation terms (≈ 194 and ≈ 323 px) confirm the images are related mainly by a horizontal camera pan.



Task 4 — CNN Scene Classification (MobileNetV2)

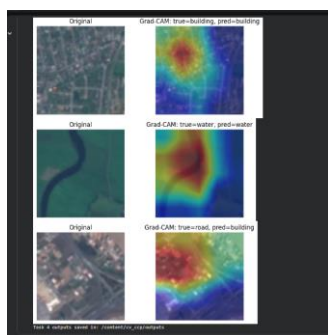
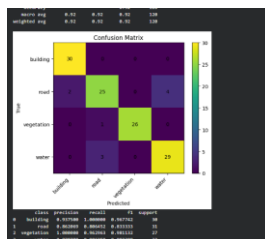
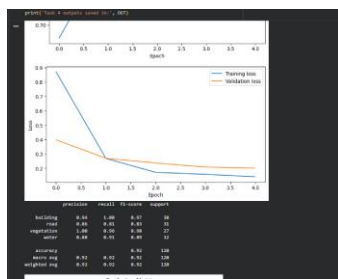
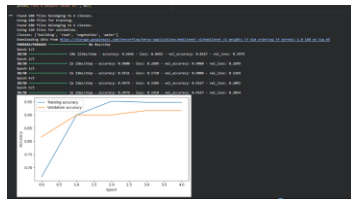
Backbone	MobileNetV2 (ImageNet weights, frozen) + GAP + Dropout(0.2) + Dense(4, softmax)
Input / augmentation	160×160; RandomFlip, RandomRotation, RandomContrast

Dataset split	600 images → 480 train / 120 validation (4 classes)
Validation accuracy	92% (target ≥ 70%)

Per-class metrics (validation set):

class	precision	recall	f1-score	support
building	0.94	1.00	0.97	30
road	0.86	0.81	0.83	31
vegetation	1.00	0.96	0.98	27
water	0.88	0.91	0.89	32
accuracy / macro avg	0.92	0.92	0.92	120

Confusion matrix (rows = true, columns = predicted): building 30/30 correct; vegetation 26/27; water 29/32 (3 → road); road 25/31 (2 → building, 4 → water). Road is the weakest class, most often confused with water.



6. Failure Analysis

I identify at least two failure modes per task below, with the causes I observed in my run.

Task	Failure mode 1	Failure mode 2	Cause / fix
T1	Extra edges from roadside vegetation and the guard-rail were picked up as lines.	Faint/worn lane markings missed at higher Canny thresholds.	Texture & threshold sensitivity; use ROI masking, adaptive thresholds, line-angle

Task	Failure mode 1	Failure mode 2	Cause / fix
			filtering.
T2	Repeated architectural motifs (arches, tilework) produce some ambiguous matches.	Fewer reliable matches where the two viewpoints differ strongly.	Repetitive texture & wide baseline; stricter ratio, RANSAC geometric check, more overlap.
T3	Faint seam/exposure step visible where frames overlap (simple 50/50 alpha blend).	Moving foreground objects would cause ghosting (seen when the Taj Mahal set with tourists was tried).	Exposure diff & parallax; multi-band blending, exposure comp., cylindrical warp.
T4	Road confused with water (4 road tiles predicted water; 3 water predicted road).	Slight train>val accuracy gap indicates mild overfitting on 150 images/class.	Similar aerial texture & limited data; more images, fine-tune deeper layers.

7. Classical vs. Deep Learning Trade-offs

Criterion	Classical CV (Tasks 1–3)	Deep learning (Task 4)
Speed	Fast on CPU; runs per-frame without training.	Training benefits from GPU (here ~13s first epoch, ~1s/epoch after); inference is fast with MobileNetV2.
Accuracy	Strong when assumptions hold (clear edges, sufficient overlap): inlier ratios >0.94 here.	Handles complex appearance/semantics: 92% validation accuracy across 4 classes.
Interpretability	Transparent — edges, keypoints, matches and homographies can be inspected directly.	Opaque by default; Grad-CAM exposes which regions drove each prediction.
Data requirement	Works with a few images and needs no labels.	Needs labelled data (600 images here) and careful validation to avoid overfitting.

8. Limitations & Future Work

My pipeline is a course-level prototype and has clear limitations. Edge detection is sensitive to shadows, rain, motion blur, and night scenes. SIFT matching and homography stitching assume sufficient overlap and mostly planar scenes or pure camera rotation; moving objects and parallax cause ghosting. I trained the classifier on aerial EuroSAT land-cover imagery, so it would not transfer directly to a ground-level robot camera without retraining on ground-level data. With more time I would improve the system by:

- Adding a road-region mask and line-angle filtering to stabilise Task 1.
- Using multi-band blending and cylindrical warping for seamless panoramas in Task 3.
- Training Task 4 on a ground-level outdoor dataset and fine-tuning deeper MobileNetV2 layers.
- Balancing classes and adding more images to reduce the road↔water confusion.
- Benchmarking frame rate and memory for real-time deployment on a robot.

9. Dataset and AI-Assistance Disclosure

Image sources. I cite all images in Section 4. For Task 1 (Murree road) and Task 2 (Wazir Khan Mosque, Lahore, by Danish Jaffry) I used photographs from Unsplash under its free-to-use license. For Task 3 I used a sample overlapping image sequence from the open-source GitHub repository Avinash793/panoramic-image-stitching (images only; I implemented my stitching pipeline independently). For Task 4 I used the EuroSAT satellite land-cover dataset (Helber et al., 2019). I did not self-capture any images.

AI assistance. I used AI assistance (Anthropic's Claude) to help structure and debug the code and to draft the wording of this report. I reviewed and verified the final code, dataset selection, experiments, execution, outputs, and interpretations myself. I make this disclosure in accordance with the course academic-integrity rules, which require declaring any LLM use.

Academic-integrity affirmation. This submitted work is my own; I have cited all external sources; I did not engage in any unauthorised collaboration or code-sharing between students; and I did not use any pre-built panorama API (e.g., cv2.Stitcher) — I implemented the stitching pipeline manually.